

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3



Code of Conduct:

*I **B Sandhya(192311292)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Testing as an Engineering Activity in a Banking Application

1. Introduction

Testing as an engineering activity means following a systematic and planned approach to identify defects and improve software quality. In banking applications, unstructured testing may cause transaction failures, incorrect balances, and security issues. Therefore, the team should replace ad-hoc testing with a structured testing process.

2. Systematic Test Design

The team should prepare a test plan based on the requirements. Test cases should cover deposits, withdrawals, fund transfers, and balance checking. Each test case must include inputs, expected results, and actual results. Testers should report defects, fix them, and perform retesting.

3. Testing Tools and Standards

Testing tools help improve efficiency and reduce manual effort.

- **Selenium:** Automates web application testing.
- **Postman:** Tests banking APIs.
- **JMeter:** Tests application performance under heavy traffic.
- **Jira:** Tracks and manages defects.

The team can follow **ISO/IEC/IEEE 29119** guidelines for software testing processes and documentation. Proper documentation helps maintain consistency and traceability.

4. Techniques to Replace Ad-hoc Testing

The following techniques can be used:

1. **Equivalence Partitioning:** Divides input data into valid and invalid groups.
2. **Boundary Value Analysis:** Tests minimum and maximum transaction limits.
3. **Decision Table Testing:** Tests combinations of conditions, such as valid PIN and sufficient balance.
4. **Regression Testing:** Ensures that software changes do not affect existing features.
5. **Automated Testing:** Executes repetitive test cases automatically.

5. Practical Example

Consider a fund transfer feature in a banking application.

Test Condition	Expected Result
Valid account and sufficient balance	Transfer successful
Insufficient balance	Transaction rejected
Invalid beneficiary account	Error message displayed
Zero transaction amount	Transaction rejected
Network failure during transfer	No duplicate debit occurs

Using boundary value analysis and equivalence partitioning, testers can identify errors in transaction amounts and account details. Regression testing ensures that updates do not affect existing banking operations.

6. Benefits and Conclusion

An engineering-based testing approach improves defect detection, increases test coverage, reduces failures, and saves time through automation. It also improves security, reliability, and customer trust. Thus,

systematic test design, suitable tools, recognized standards, and structured testing techniques help deliver a **reliable and high-quality banking application**.

2. Structured Testing Process for an ATM System

1. Introduction

A structured testing process is a systematic approach to testing software throughout the Software Development Life Cycle (SDLC). In an ATM system, improper testing may cause incorrect withdrawals, wrong balance updates, and transaction failures. Therefore, testing should be performed at every SDLC phase to improve software quality.

2. Testing Process Across SDLC Phases

1. Requirement Analysis:

Identify ATM requirements such as cash withdrawal, balance checking, PIN verification, and receipt generation. This helps testers understand what needs to be tested.

2. Test Planning:

Prepare a test plan that defines the testing scope, resources, schedule, tools, and responsibilities. This ensures testing is organized and consistent.

3. Test Case Design:

Prepare test cases for valid and invalid PINs, sufficient and insufficient balances, and available or unavailable cash. This improves test coverage.

4. Test Environment Setup:

Configure the ATM software, banking database, card reader, and cash dispenser in a test environment. This helps identify environment-related issues.

5. Test Execution:

Execute the test cases and compare actual results with expected results. Report any failures as defects.

6. Defect Reporting and Retesting:

Developers fix identified defects, and testers retest the affected functionality to verify the fixes.

7. Regression Testing:

Repeat relevant existing test cases after changes to ensure that previously working ATM functions are not affected.

8. Test Closure:

Prepare a test summary report containing test results, defects, test coverage, and remaining risks. This helps evaluate the overall quality of the system.

3. Practical Example: ATM Withdrawal Functionality

Consider an ATM that allows customers to withdraw money using their debit cards.

Test Case	Input/Condition	Expected Result
Valid withdrawal	Correct PIN and sufficient balance	Cash dispensed and balance updated
Invalid PIN	Incorrect PIN	Error message displayed
Insufficient balance	Withdrawal exceeds account balance	Transaction rejected
Invalid amount	Zero or unsupported amount	Error message displayed
Insufficient ATM cash	ATM does not have enough cash	Withdrawal rejected without debiting the account
Network failure	Connection lost during withdrawal	Transaction handled safely without duplicate debit

These test cases help identify defects in PIN verification, balance calculation, cash dispensing, and transaction processing.

4. How Process Consistency Reduces Defects

A structured testing process reduces defects by ensuring that test cases are prepared, executed, and documented consistently. It prevents important scenarios from being missed, helps identify defects early, and supports systematic retesting and regression testing. It also improves communication between testers and developers.

5. Conclusion

Implementing a structured testing process across all SDLC phases improves the reliability, security, and accuracy of an ATM system. It reduces transaction failures, prevents incorrect balance updates, and increases customer confidence. The expected outcome is a **consistent, secure, and high-quality ATM system** that performs withdrawals correctly under normal and exceptional conditions.

3.Errors, Defects, and Failures in a Web Application

1. Introduction

In software testing, errors, defects, and failures are related concepts used to identify and understand software problems. A web application showing mismatches between expected and actual outputs during login and transactions may contain defects that lead to failures.

2. Basic Testing Definitions

1. Error:

An error is a human mistake made during software development.

Example: A developer mistakenly writes incorrect logic for validating a user's password.

2. Defect:

A defect is a flaw in the software caused by an error in requirements, design, or coding.

Example: The login program accepts an incorrect password because the validation logic is wrong.

3. Failure:

A failure occurs when the software does not perform its expected function during execution.

Example: A user enters an incorrect password, but the website allows access to the account.

3. Relationship Between Error, Defect, and Failure

These concepts are related as follows:

Human Error → Software Defect → Software Failure

For example:

- **Error:** A developer writes incorrect transaction calculation logic.
- **Defect:** The application contains a bug that deducts ₹500 instead of ₹200.
- **Failure:** When the customer withdraws ₹200, the account balance decreases by ₹500.

A defect may cause a failure when the affected software is executed under particular conditions.

4. Examples in Login and Transaction Operations

Operation	Defect	Failure
Login	Incorrect password validation logic	Invalid users can log in
Money Transfer	Incorrect balance calculation	Wrong amount is deducted
Payment	Missing duplicate transaction check	Customer is charged twice
Login	Incorrect error-message logic	Wrong message is displayed

5. Effects of Improper Understanding

If testers confuse errors, defects, and failures, they may report problems incorrectly. This can cause communication gaps between testers and developers, delay defect fixing, and lead to incomplete testing. For example, reporting only “login not working” does not clearly identify the observed problem or its possible cause.

6. Improving Defect Reporting Clarity

Defect reports should include the following details:

- **Defect ID:** Unique identification number.
- **Title:** Short description of the problem.
- **Steps to Reproduce:** Actions required to reproduce it.
- **Expected Result:** What should happen.
- **Actual Result:** What actually happened.
- **Severity and Priority:** Impact of the problem and urgency of fixing it.

- **Evidence:** Screenshots, logs, or error messages.

Example:

- **Title:** Incorrect amount deducted during money transfer.
- **Steps:** Log in, transfer ₹200, and check the account balance.
- **Expected Result:** Balance decreases by ₹200.
- **Actual Result:** Balance decreases by ₹500.
- **Severity:** High, because the defect affects financial transactions.

7. Conclusion

Understanding errors, defects, and failures helps testers identify software problems accurately. Clear defect reports improve communication, speed up debugging, and reduce repeated issues. Therefore, applying these concepts systematically helps deliver a **reliable, accurate, and high-quality web application**.

4. Software Testing Principles

1. Defect Clustering:

Most defects are found in a few modules. Testers should focus more on these high-defect modules.

2. Testing Shows the Presence of Defects:

Testing can identify defects but cannot prove that software is completely defect-free.

3. Early Testing:

Start testing early in the SDLC to find defects and reduce fixing costs.

4. Risk-Based Testing:

Test critical modules, such as payment and login, before low-risk modules.

Test Case Prioritization

Test cases should be prioritized based on defect frequency, business importance, and risk.

- **High Priority:** Payment failures, login errors.
- **Medium Priority:** Profile updates.
- **Low Priority:** Font size and minor UI issues.

Real-Time Example

In an online banking application, most defects occur in the money transfer module. Testers should prioritize fund transfers, balance calculations, and duplicate transaction checks before testing minor UI features.

Conclusion

Applying software testing principles helps detect critical defects early, saves time and cost, improves test coverage, and increases software reliability.

5.1. Defect Classification

Defects originate from mistakes in requirements, design, coding, or testing. They can be classified as:

- **Functional Defects:** A feature does not work correctly.
- **Performance Defects:** The application responds slowly.
- **Security Defects:** Unauthorized access is possible.
- **UI Defects:** Problems occur in the user interface.

2. Defect Repository

A defect repository stores defect details, including defect ID, type, severity, root cause, affected module, and resolution. It helps teams track and analyze previously identified defects.

3. Using Repository Data

Testers can analyze previous defect records to identify frequently failing modules and prepare additional test cases. For example, if payment failures occur repeatedly, testers should create more test cases for payment validation and transaction processing.

4. Preventing Recurring Defects

- Analyze the root cause of repeated defects.
- Update test cases based on previous failures.
- Perform regression testing before every release.
- Share defect information among developers and testers.
- Review code and improve development practices.

5. Conclusion

Effective defect classification and repository usage help identify recurring problems, improve test coverage, reduce repeated defects, and deliver **more reliable, secure, and high-quality software**.

6.1. Root Cause Identification

Check crash logs, code, and network connections to identify why the mobile banking application crashes during fund transfers.

2. Systematic Testing

- **Early Testing:** Find defects during development.
- **Risk-Based Testing:** Prioritize fund transfer operations.
- **Boundary Testing:** Test minimum and maximum transfer amounts.
- **Regression Testing:** Ensure fixes do not affect existing features.
- **Performance Testing:** Check application behavior under heavy usage.

3. Practical Example

Test Condition	Expected Result
Sufficient balance	Transfer successful
Insufficient balance	Transaction rejected
Network failure	No duplicate debit
Invalid account	Error message displayed

4. Improvements

- Automate repeated test cases.
- Analyze the root cause of defects.
- Test different devices and network conditions.
- Perform regression testing before release.

5. Conclusion

Systematic testing helps identify defects, reduce crashes, prevent duplicate transactions, and improve the **security, accuracy, and reliability** of mobile banking applications.

7.1. Testing vs Debugging

Testing: Testing is the process of executing software to identify defects by comparing actual and expected results.

Debugging: Debugging is the process of finding the root cause of a defect and fixing it.

2. Roles in Software Quality

- **Testing:** Detects defects and verifies software functionality.
- **Debugging:** Identifies the cause of defects and corrects them.

3. Practical Example

In a banking application, testing identifies that ₹500 is deducted instead of ₹200 during a transfer. Debugging helps developers find and correct the incorrect calculation logic.

4. Effects of Improper Usage

Confusing testing with debugging may lead to incomplete defect detection, incorrect fixes, and repeated failures. Fixing a defect without retesting may introduce new problems.

5. Structured Approach

1. Test the application and identify defects.

2. Report defects with expected and actual results.
3. Debug the code to find the root cause.
4. Fix the defect and perform retesting.
5. Conduct regression testing to ensure existing features work correctly.

6. Conclusion

Testing identifies defects, while debugging finds and fixes their causes. Using both systematically improves software quality, reduces repeated defects, and increases reliability.

8.1. Testing Strategy

Cross-browser testing ensures that an e-commerce application works correctly across different browsers such as Chrome, Firefox, and Edge.

- Test important features like login, product search, cart, and payment.
- Prepare test cases for each browser.
- Compare actual results with expected results.
- Perform regression testing after fixing defects.

2. Defect Origins

Defects may occur due to differences in browser rendering, JavaScript support, CSS compatibility, and browser versions.

Example: A checkout button works in Chrome but does not work in Firefox due to a JavaScript compatibility issue.

3. Systematic Testing

Systematic testing helps identify browser-specific defects, improves test coverage, and ensures consistent user experience across browsers.

4. Tools and Techniques

- **Selenium:** Automates browser testing.
- **BrowserStack:** Tests applications across different browsers and devices.
- **Equivalence Partitioning:** Tests representative input groups.
- **Regression Testing:** Ensures fixes do not affect existing features.

5. Conclusion

Cross-browser testing using suitable tools and systematic techniques reduces compatibility issues and improves the **reliability, usability, and quality** of an e-commerce application.

9.1. Defect Classification

Defect classification is the process of grouping software defects based on their type and impact.

- **Functional Defects:** Salary calculation does not follow requirements.
- **Logical Defects:** Incorrect formulas are used for salary calculations.
- **Calculation Defects:** Wrong deductions or allowances are calculated.
- **Boundary Defects:** Incorrect salary is calculated at tax or deduction limits.

2. Importance of Defect Classification

Classification helps testers identify defect patterns, select suitable test cases, and prioritize critical salary calculation errors.

3. Practical Example

If an employee's basic salary is ₹30,000 and the system incorrectly calculates deductions, the final salary will be wrong. Testers should verify allowances, deductions, taxes, and net salary against expected results.

4. Prevention of Future Defects

- Review code and salary calculation formulas.
- Prepare test cases for different salary ranges.
- Perform boundary value testing.
- Conduct regression testing after fixes.
- Maintain a defect repository to track repeated errors.

5. Conclusion

Defect classification and systematic testing help identify logical errors, prevent recurring salary calculation problems, and improve the **accuracy, reliability, and quality** of the payroll system.

10.1. Testing as an Engineering Activity

Testing as an engineering activity means following a systematic and planned process instead of executing random tests.

2. Importance of Structured Test Case Design

- Defines clear testing objectives.
- Covers valid and invalid inputs.
- Improves test coverage.
- Reduces the chance of missing defects.
- Makes testing repeatable and consistent.

3. Practical Example

For a login application:

Test Case	Expected Result
Valid username and password	Login successful
Invalid password	Error message displayed
Empty login fields	Validation message displayed
Incorrect username	Login rejected

Structured test cases help identify defects that random testing may miss.

4. Suitable Testing Techniques

- **Equivalence Partitioning:** Divides inputs into valid and invalid groups.
- **Boundary Value Analysis:** Tests minimum and maximum input values.
- **Decision Table Testing:** Tests different combinations of conditions.
- **Regression Testing:** Checks existing features after changes.
- **Automation Testing:** Executes repetitive tests efficiently.

5. Conclusion

Structured test case design improves defect detection, test coverage, and consistency. Applying systematic testing techniques helps deliver **reliable and high-quality software**.

11.1. Testing Principles

Testing principles help identify defects systematically and ensure software quality. In a healthcare application, data inconsistency may cause incorrect patient records, prescriptions, or test results.

2. How Defects Originate

Defects may occur due to incorrect coding, database errors, improper data validation, or communication failures between modules.

3. Systematic Testing for Data Integrity

- **Integration Testing:** Checks data exchange between modules.
- **Validation Testing:** Ensures patient details are entered correctly.
- **Regression Testing:** Verifies that changes do not affect existing functions.
- **Database Testing:** Checks data accuracy and consistency.

Example: If a patient's updated blood group appears incorrectly in another module, integration and database testing can help identify the defect.

4. Defect Tracking and Reporting

- Assign a unique ID to every defect.
- Record expected and actual results.
- Mention severity and affected modules.
- Maintain a defect repository.
- Retest after fixing defects.

5. Conclusion

Systematic testing and proper defect tracking help maintain data integrity, prevent repeated errors, and improve the **accuracy, reliability, and safety** of healthcare applications.

12.1. Defect Repository

A defect repository stores information about software defects, including their type, severity, root cause, affected module, and solution.

2. How Repository Data Supports Test Design

Testers analyze previous defects to identify risky modules and prepare additional test cases. This improves test coverage and helps detect defects early.

3. Identifying Defect Patterns

For example, if payment-related defects occur repeatedly across releases, testers can identify the payment module as a high-risk area and focus more testing on it.

4. Preventing Recurring Defects

- Analyze the root cause of repeated defects.
- Update test cases using previous defect data.
- Perform regression testing before every release.
- Conduct code reviews.
- Maintain and update the defect repository regularly.

5. Conclusion

Effective use of a defect repository helps identify defect patterns, prevent repeated errors, improve test coverage, and deliver **more reliable, high-quality software**.

13.1. Testing Principles

Software testing principles help identify defects and ensure system reliability. A web-based learning system may fail during peak usage due to insufficient performance testing and incomplete test coverage.

2. Testing Gaps

- **Insufficient Performance Testing:** Heavy user traffic is not tested.
- **Incomplete Test Coverage:** Important scenarios are missed.
- **Lack of Risk-Based Testing:** Critical features are not prioritized.
- **Insufficient Regression Testing:** Changes introduce new defects.

3. Importance of Test Planning and Coverage

Test planning defines objectives, resources, schedules, and testing techniques. Test coverage ensures that important scenarios are tested, including simultaneous logins, video streaming, and online examinations.

4. Practical Example

During an online examination, thousands of students log in simultaneously, causing the website to crash. Load testing and stress testing can identify server capacity problems before release.

5. Improvements

- Perform load and stress testing using JMeter.
- Test realistic peak-user scenarios.
- Prioritize critical features.
- Automate repetitive test cases.
- Monitor performance and fix bottlenecks.
- Conduct regression testing before releases.

6. Conclusion

Proper test planning, sufficient test coverage, and performance testing help identify bottlenecks early, prevent system crashes, and improve the **reliability, performance, and user experience** of the learning system.

14.1. Role of Tester in SDLC

Testers play an important role in every phase of the Software Development Life Cycle (SDLC). They review requirements, design test cases, execute tests, report defects, and verify fixes.

2. Importance of Early Involvement

- Identifies requirement errors early.
- Reduces defect-fixing costs.
- Improves test coverage.
- Prevents major failures in later stages.
- Improves software quality.

3. Practical Example

In an online banking application, testers review the fund transfer requirements during the requirement phase. They identify that duplicate transactions are not addressed. Developers add proper transaction handling before implementation, preventing future financial errors.

4. Team Collaboration Improvements

- Include testers in requirement meetings.
- Conduct regular discussions between developers and testers.
- Review requirements and designs together.
- Use defect-tracking tools.
- Perform continuous testing throughout development.

5. Conclusion

Early tester involvement helps identify defects sooner, reduces development costs, improves collaboration, and delivers **reliable, high-quality software**.

15.1. Testing Principles

Incomplete test coverage means some features or scenarios are not tested. Testing principles help identify missing test cases and improve software quality.

2. Importance of Test Coverage

- Identifies defects in different scenarios.
- Ensures important requirements are tested.
- Reduces the risk of failures.
- Improves software reliability.

3. Practical Example

In a banking application, money transfer works correctly when the balance is sufficient but fails when the balance is insufficient. Testing only successful transactions misses this defect.

4. Improving Test Case Design

- Prepare test cases for valid and invalid inputs.
- Test normal, boundary, and exceptional conditions.
- Map test cases to requirements.
- Perform regression testing after changes.

5. Validation Techniques

- **Equivalence Partitioning:** Divide inputs into valid and invalid groups.
- **Boundary Value Analysis:** Test minimum and maximum values.

- **Decision Table Testing:** Test different combinations of conditions.
- **Integration Testing:** Verify data exchange between modules.

6. Conclusion

Proper test coverage and systematic test design help detect hidden defects, reduce failures, and improve the **reliability and quality** of software.

16.1. Testing Concepts

Defect reporting is the process of recording software defects clearly so developers can identify and fix them. In a logistics application, poor reporting may delay the identification of delivery tracking and shipment calculation errors.

2. Importance of Clear Defect Definitions

- **Error:** A human mistake during development.
- **Defect:** A flaw in the software.
- **Failure:** Incorrect software behavior during execution.

Understanding these terms helps testers report problems accurately and track them effectively.

3. Practical Example

If a logistics application displays the wrong delivery date, the tester should record the expected date and actual date instead of reporting only “delivery problem.”

4. Structured Defect Reporting

A defect report should include:

- Defect ID and title.
- Steps to reproduce.
- Expected and actual results.
- Severity and priority.
- Screenshots or error logs.
- Defect status.

5. Improvements

- Follow a standard defect-reporting template.
- Use tools such as Jira to track defects.
- Train testers in clear reporting.
- Review defect reports regularly.
- Maintain a defect repository.

6. Conclusion

Clear defect definitions and structured reports improve communication, speed up defect fixing, reduce repeated errors, and increase the **reliability and quality** of the logistics application.

17.1. Process-Based Testing Approach

Follow a standard testing process for all modules, including test planning, test case design, execution, and defect reporting.

2. How Standardized Processes Improve Quality

Standardized processes ensure uniform testing, reduce errors, improve test coverage, and increase software reliability.

3. Example

In an online shopping application, all modules (login, payment, and orders) follow the same testing procedure and defect reporting format.

4. Methods to Ensure Consistency

- Use standard testing templates.
- Follow common testing guidelines.
- Conduct regular reviews.
- Use automation tools.
- Provide training to testers.

18.1. Testing Principles

The system failed because edge cases were not properly tested before deployment. Insufficient test coverage and limited test data can cause such failures. Testers should verify normal, invalid, and extreme input values.

2. Boundary Testing

Boundary Value Analysis checks values at the minimum and maximum limits of an input range. It helps identify errors that occur at the edges of acceptable values.

3. Example

If an application accepts ages from 18 to 60, test the values **17, 18, 19, 59, 60, and 61**. This helps identify errors in handling valid and invalid age limits.

4. Improvements in Test Design

- Include boundary value testing.
- Test valid and invalid inputs.
- Use equivalence partitioning.
- Increase test coverage.

- Perform regression testing before deployment.
- Automate repeated test cases to reduce human errors.

19.1. Defect Repository Concepts

A defect repository is a centralized database used to record, store, and track software defects. The company should maintain a common repository to manage all defects systematically.

2. How Documentation Improves Defect Management

Proper documentation helps testers identify, track, assign, and resolve defects easily. It avoids duplicate reports, improves communication, and ensures that no defect is missed.

3. Example

In an online shopping application, a payment failure is recorded with a defect ID, description, severity, steps to reproduce, screenshots, assigned developer, and current status. The team can track the defect until it is resolved.

4. Improvements in Documentation Practices

- Use a centralized defect tracking tool.
- Record complete details for every defect.
- Assign unique IDs to defects.
- Update defect status regularly.
- Avoid duplicate defect reports.
- Conduct regular defect reviews and maintain proper records.

20.1. Software Testing Principles

Software testing is the process of identifying defects and verifying whether a system meets its requirements. In a banking system, improper testing can cause transaction failures, incorrect balances, and duplicate payments.

To redesign the testing approach, the team should follow these principles:

- **Early Testing:** Start testing during the requirement and design phases to identify defects early.
- **Exhaustive Testing is Impossible:** Test important scenarios, including normal transactions, invalid inputs, and exceptional conditions.
- **Defect Clustering:** Focus more on modules where defects occur frequently, such as payment processing.
- **Testing is Context-Dependent:** Use strict testing procedures because banking systems handle sensitive financial transactions.
- **Absence of Errors Fallacy:** A system must meet user requirements, not just be free from defects.

2. How Structured Testing Improves Quality

Structured testing follows a systematic process of test planning, test case design, test execution, defect reporting, and regression testing.

It improves software quality by:

- Identifying defects systematically.
- Ensuring better test coverage.
- Reducing transaction failures.
- Improving data accuracy and security.
- Preventing previously fixed defects from recurring.
- Increasing system reliability and customer confidence.

3. Example of Testing a Banking System

Consider a banking application that transfers money between two accounts.

Scenario: A customer transfers ₹5,000 from Account A to Account B.

Expected Result:

- Account A balance decreases by ₹5,000.
- Account B balance increases by ₹5,000.
- The transaction is recorded successfully.

Test Cases:

Test Case	Input/Condition	Expected Result
Valid transaction	Sufficient balance	Transfer succeeds
Insufficient balance	Balance is lower than the transfer amount	Transaction is rejected
Invalid account	Incorrect account number	Error message is displayed
Duplicate request	Same transaction request is submitted twice	Money is not transferred twice
Network failure	Connection fails during transfer	Transaction remains consistent
Zero amount	Transfer amount is ₹0	Invalid transaction is rejected

These test cases help identify transaction-processing errors before deployment.

4. Techniques to Enhance Defect Detection

1. **Unit Testing:** Test individual components, such as balance calculation and transaction validation.
2. **Integration Testing:** Verify communication between account, payment, and transaction modules.
3. **System Testing:** Test the complete banking application under realistic conditions.

4. **Boundary Value Analysis:** Test minimum and maximum transaction limits and values just outside those limits.
5. **Regression Testing:** Retest the system after fixing defects to ensure existing features still work.
6. **Automation Testing:** Automate repeated transactions and validation tests to improve efficiency.
7. **Load Testing:** Simulate many users performing transactions simultaneously to identify performance problems.
8. **Security Testing:** Check authentication, authorization, and protection against unauthorized transactions.
9. **Defect Tracking:** Record defects with unique IDs, severity, reproduction steps, and resolution status.

Conclusion

Software testing principles help identify weaknesses in banking systems and prevent repeated transaction failures. Structured testing improves accuracy, reliability, security, and overall software quality. By combining unit, integration, system, regression, automation, and security testing, the company can detect defects early and deliver a dependable banking application.